МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена Трудов ого Красного Знамени федеральное государственное бюджетное образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Кафедра Программная инженерия

ОТЧЕТ

по лабораторной работе №7

по дисциплине «Введение в информационные технологии»

Тема: «СОЗДАНИЕ ЧАТА С ИСПОЛЬЗОВАНИЕМ WEBSOCKETS (FASTAPI + AIOHTTP)»

Выполнил: студент группы БВТ2501

Лебедев Даниил Владимирович

Москва, 2026

Цель работы

Научиться реализовывать простой веб-чат с использованием WebSockets на базе FastAPI и AIOHTTP. Изучить основы асинхронного взаимодействия между клиентом и сервером.

Ход работы

0. Копируем код из приложения

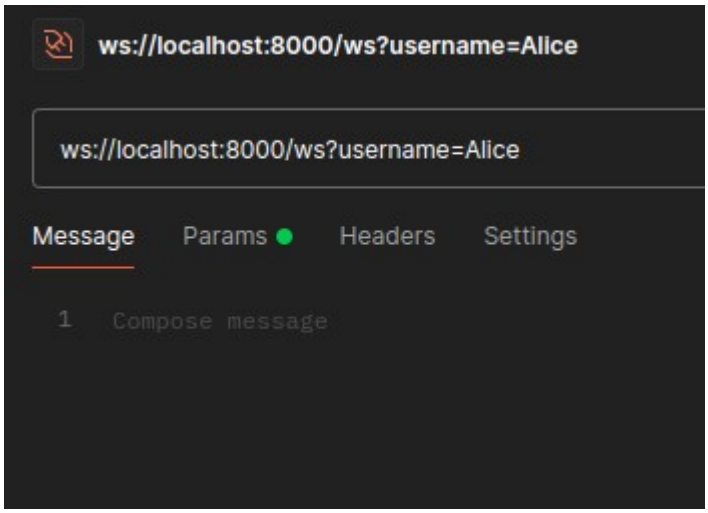
```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from fastapi.responses import HTMLResponse
from fastapi.staticfiles import StaticFiles
from pathlib import Path
import uvicorn

app = FastAPI()

html_path = Path(__file__).parent / "templates"
app.mount("/static", StaticFiles(directory=html_path), name="static")

clients = []


@app.get("/")
async def get():
    html = (html_path / "chat.html").read_text()
    return HTMLResponse(content=html, status_code=200)


@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()

    clients.append(websocket)

    try:
        while True:
            data = await websocket.receive_text()
            for client in clients:
                if client != websocket:
                    await client.send_text(data)
    except WebSocketDisconnect:
        clients.remove(websocket)

if __name__ == "__main__":
    uvicorn.run("main:app", reload=True)
```

```html
<!DOCTYPE html>
<html>
<head>
  <title>WebSocket Chat</title>
</head>
<body>
  <h1>Чат на WebSockets</h1>
  <input type="text" id="messageInput" autocomplete="off"/>
  <button onclick="sendMessage()">Отправить</button>
  <ul id="messages"></ul>
  <script>
    const ws = new WebSocket("ws://localhost:8000/ws");
    ws.onmessage = function(event) {
      const messages = document.getElementById("messages");
      const message = document.createElement("li");
      message.textContent = event.data;
      messages.appendChild(message);
    };
    function sendMessage() {
      const input = document.getElementById("messageInput");
      ws.send(input.value);
      input.value = '';
    }
  </script>
</body>
</html>
```

1. Добавление username

```python
@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket, username: str = Query(None)):

  if not username or username.strip() == "":
    await websocket.close(code=1008)
    return
```

через postman видим, что всё работает



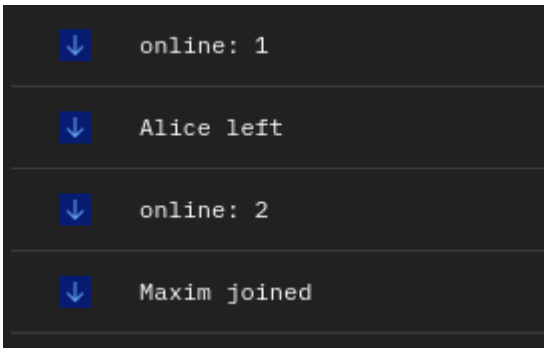

```
INFO:     127.0.0.1:57708 - "WebSocket /ws?username=Alice" [accepted]
INFO:     connection open
```

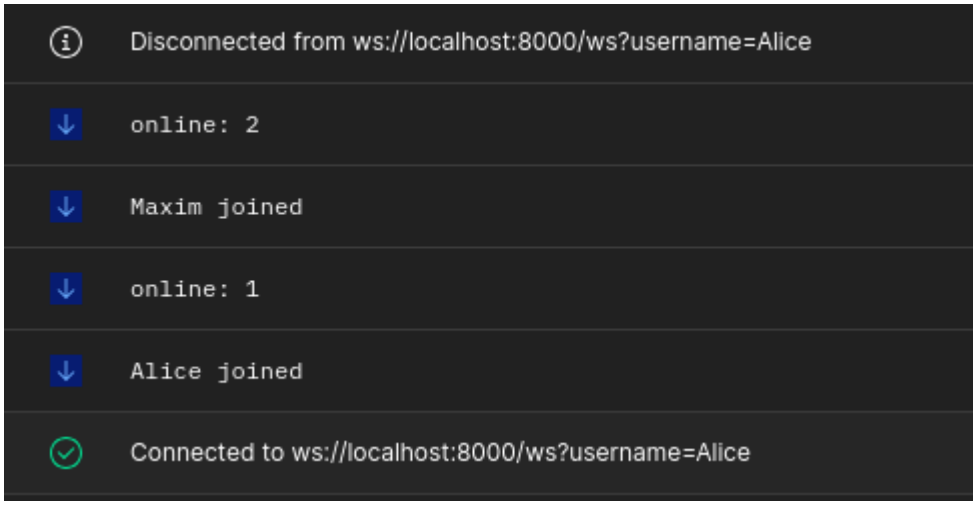
2. Системные сообщения

```python
async def broadcast(message: str):
    for client in clients:
        await client.send_text(message)


@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket, username: str = Query(None)):

    if not username or username.strip() == "":
        await websocket.close(code=1008)
        return

    await websocket.accept()
    clients.append(websocket)

    await broadcast(f"{username} joined")
    await broadcast(f"online: {len(clients)}")

    try:
        while True:
            data = await websocket.receive_text()
            for client in clients:
                if client != websocket:
                    await client.send_text(data)
    except WebSocketDisconnect:
        clients.remove(websocket)
        await broadcast(f"{username} left")
        await broadcast(f"online: {len(clients)}")
```

создаём отдельную функцию для удобства и с помощью её доставляем сообщения

Проверим с помощью postman





что и следовало ожидать

3. json

```
@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket, username: str = Query(None)):

    if not username or username.strip() == "":
        await websocket.close(code=1008)
        return

    await websocket.accept()
    clients.append(websocket)

    await broadcast({
        "type": "info",
        "user": "System",
        "text": f"{username} joined. Online: {len(clients)}",
        "ts": datetime.datetime.now().isoformat()
    })

    try:
        while True:
```

```
        raw_data = await websocket.receive_text()
        data_json = json.loads(raw_data)
        payload = {
            "type": "user",
            "user": username,
            "text": data_json["text"],
            "ts": datetime.datetime.now().isoformat()
        }
        await broadcast(payload)
    except WebSocketDisconnect:
        clients.remove(websocket)
        await broadcast({
            "type": "info",
            "user": "System",
            "text": f"{username} left. Online: {len(clients)}",
            "ts": datetime.datetime.now().isoformat()
        })

async def broadcast(data: dict):
    message = json.dumps(data)
    for client in clients:
        await client.send_text(message)
```

```
{"type": "message", "user": "Alice", "text": "message", "ts": "2026-03-09T21:32:51.011393"}
{"text": "message"}
{"type": "info", "user": "System", "text": "Maxim joined. Online: 2", "ts": "2026-03-09T21:32:35.873906"}
{"type": "info", "user": "System", "text": "Alice joined. Online: 1", "ts": "2026-03-09T21:32:34.226188"}
```

видим, что всё работает

4. Валидация сообщений

```
fclass Message(BaseModel):
    text: str = Field(..., min_length=1, max_length=200)



@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket, username: str = Query(None)):

    if not username or username.strip() == "":
        await websocket.close(code=1008)
        return

    await websocket.accept()
    clients.append(websocket)

    await broadcast({
        "type": "info",
        "user": "System",
        "text": f"{username} joined. Online: {len(clients)}",
```

```
    "ts": datetime.datetime.now().isoformat()
})

try:
    while True:
        raw_data = await websocket.receive_text()

        try:
            data_json = json.loads(raw_data)

            msg_valid = Message(**data_json)

            response_payload = {
                "type": "message",
                "user": username,
                "text": msg_valid.text,
                "ts": datetime.datetime.now().isoformat()
            }
            await broadcast(response_payload)

        except (json.JSONDecodeError, TypeError):
            await websocket.send_text(json.dumps({
                "type": "error",
                "detail": "Invalid JSON format"
            }))
        except ValidationError as e:
            await websocket.send_text(json.dumps({
                "type": "error",
                "detail": "Message is empty or too long (max 200 chars)"
            }))

except WebSocketDisconnect:
    clients.remove(websocket)
    await broadcast({
        "type": "info",
        "user": "System",
        "text": f"{username} left. Online: {len(clients)}",
        "ts": datetime.datetime.now().isoformat()
    })
```

```
{"type": "error", "detail": "Message is empty or too long (max 200 chars)"}
{"text": ""}
{"type": "message", "user": "Alice", "text": "message", "ts": "2026-03-09T21:32:51.011393"}
{"text": "message"}
{"type": "info", "user": "System", "text": "Maxim joined. Online: 2", "ts": "2026-03-09T21:32:35.873906"}
{"type": "info", "user": "System", "text": "Alice joined. Online: 1", "ts": "2026-03-09T21:32:34.226188"}
```

мы получили ошибку, что сообщение пустое

5. приватные сообщения

здесь, мы должны поменять структуру clients на любую парную, для нас выгоднее всего словари

Полный код программы:

```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect, Query
from fastapi.templating import Jinja2Templates
from fastapi.staticfiles import StaticFiles
from pathlib import Path
import uvicorn
import json
import datetime

from pydantic import ValidationError, Field, BaseModel
from fastapi import Request

app = FastAPI()

html_path = Path(__file__).parent / "templates"
templates = Jinja2Templates(directory=html_path)
app.mount("/static", StaticFiles(directory=html_path), name="static")

clients = {}


class Message(BaseModel):
    text: str = Field(..., min_length=1, max_length=200)


async def broadcast(data: dict):
    message = json.dumps(data)
    for client in clients:
        await clients[client].send_text(message)


@app.get("/")
async def get(request: Request, username: str = Query(None)):
    return templates.TemplateResponse("chat.html", {"request": request, "user": username})
```

```python
@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket, username: str = Query(None)):

    if not username or username.strip() == "":
        await websocket.close(code=1008)
        return

    await websocket.accept()
    clients[username] = websocket

    await broadcast({
        "type": "info",
        "user": "System",
        "text": f"{username} joined. Online: {len(clients)}",
        "ts": datetime.datetime.now().isoformat()
    })

    try:
        while True:
            raw_data = await websocket.receive_text()

            try:
                data_json = json.loads(raw_data)

                msg_valid = Message(**data_json)

                if msg_valid.text.startswith("/w "):
                    pred_msg = msg_valid.text.split()
                    if len(pred_msg) > 2:
                        target = pred_msg[1]
                        text = " ".join(pred_msg[2:])
                        if target in clients:
                            private_msg = {
                                "type": "private",
                                "from": username,
                                "to": target,
                                "text": text,
                                "ts": datetime.datetime.now().isoformat()
                            }

                            await clients[target].send_text(json.dumps(private_msg))

                            await websocket.send_text(json.dumps(private_msg))

                        else:

                            await websocket.send_text(json.dumps({
                                "type": "error",
                                "detail": f"User {target} not found"
```

```
                }))
                continue
            response_payload = {
                "type": "message",
                "user": username,
                "text": msg_valid.text,
                "ts": datetime.datetime.now().isoformat()
            }
            await broadcast(response_payload)

        except (json.JSONDecodeError, TypeError):
            await websocket.send_text(json.dumps({
                "type": "error",
                "detail": "Invalid JSON format"
            }))
        except ValidationError as e:
            await websocket.send_text(json.dumps({
                "type": "error",
                "detail": "Message is empty or too long (max 200 chars)"
            }))

    except WebSocketDisconnect:
        del clients[username]
        await broadcast({
            "type": "info",
            "user": "System",
            "text": f"{username} left. Online: {len(clients)}",
            "ts": datetime.datetime.now().isoformat()
        })

if __name__ == "__main__":
    uvicorn.run("main:app", reload=True)
```

проверяем

```
{"type": "private", "from": "Alice", "to": "Maxim", "text": "message 1", "ts": "2026-03-09T21:50:31.645336"}
{"text": "/w Maxim message 1"}
{"type": "private", "from": "Alice", "to": "Maxim", "text": "message", "ts": "2026-03-09T21:50:25.892616"}
{"text": "/w Maxim message"}
{"type": "info", "user": "System", "text": "Maxim joined. Online: 2", "ts": "2026-03-09T21:50:23.393500"}
{"type": "info", "user": "System", "text": "Alice joined. Online: 1", "ts": "2026-03-09T21:50:21.473220"}
Connected to ws://localhost:8000/ws?username=Alice
```

Сообщение доставлено

Заключение

Мы научились реализовывать простой веб-чат с использованием WebSockets на базе FastAPI и AIOHTTP. Изучили основы асинхронного взаимодействия между клиентом и сервером.

Гитхаб
https://github.com/Bulbanator123/labsmtucivvit